Institute for Computer Science VI, Autonomous Intelligent Systems, University of Bonn

Dr. N. Goerke
Friedrich-Hirzebruch-Allee 8, 53115 Bonn, Tel: +49 228 73-4167
E-Mail: goerke (at) ais.uni-bonn.de
http://www.ais.uni-bonn.de/WS/4204_L_NN.html

Exercises for module Technical Neural Networks (MA-INF 4204), WS26/27

Assignments Sheet 2b, due: Tuesday 27.10.2026

20.10.2026

Assignment 7 (1 Point)

Write down the δ-rule.
Explain **each** part of the formula with a short sentence.

Assignment 8 (2 Points)

A given MLP with two hidden layers, N-H1-H2-M MLP shall be replaced by a second MLP (N-H-M) with only one single hidden layer, but (almost) the same number of weights.
Derive a formula for the number H of neurons in that hidden layer, and apply your formula for H to replace a 5-21-30-4 MLP. Hint: please do not forget the BIAS.

Assignment 9 (3 Points)

Prove in a strict formal way, **analytically**, that a simple Perzeptron without a hidden layer (two input neurons, BIAS, one output Neuron with step function) is not capable to implement the Boolean function XOR.

Assignment 10b (2 Points)

Derive a new learning rule * for an output neuron of a Perceptron.
Start from the new objective function (cost function, error function) E^* and derive the new learning rule in the way presented in the lecture. Write down all calculation steps, and explicitly write down the formulas for calculating the δ^* in output-layer.

$$^pE^*(w_{ij}) = \frac{1}{2}\sum_{m=0}^{M}\left(\,^p\hat{y}_m \;-\; ^py_m\,\right)^6$$

Assignment 11b (2 Points)

Assignment 11 has to be operated using AI help.
Report which AI-System helped you, which prompts you have decided to use. What the percentage did the AI-system contribute to your solution? In case you had to re-set your AI-session to get an answer, report why, when and how often. Have you discarded some of the AI-generated answers? If yes, Why?

Show by calculation that the first derivatives of the two typical transferfunctions for MLPs (*tanh* and *logistic function*) can be expressed by the transferfunctions themselves.

Assignment 12b (2 Points)

Show by calcluation that the transferfunction $tanh(z)$ is identical to the shifted and rescaled Fermi-function (also called logistic function) $f_{\log}(z) = \frac{1}{1+e^{-z}}$.

Programming-Assignment PA-A (10 Points, due 3.11.2026)

Implement in Python a Multi-Layer-Perzeptron including the learning rule Backpropagation of Error and test your program. Do not use libraries that have pre-implemented neural network structures.

Implement a Multi Layer Perzeptron

Your program must be capable of setting the structure of the network, number of layers (maximum 4), number of neurons per layer (maximum 1000), transferfunction separately for each layer (*tanh* or *logistic* or *identity*). It is O.K. to set these parameters directly within the source-code, please do not implement a user interface for that.

Initialize the weights to random values between -2.0 and $+2.0$, make sure, that the random number generator is under your control, and that you can reproduce your results. Set/initialize the random number generator explicitly (random seed).

Implement Backpropagation of Error

Implement the 7 steps of the Backpropagation of Error Algorithm (from the lecture). Your program shall read training patterns (input ${}^p x_n$, teacher ${}^p \hat{y}_m$) from a file `training_data.txt` with up to $P = 1000$ patterns. Use the sum over quadratic differences as error function. Allow to set different learning rates η for the different layers.

Calculate the error in every training step, and print it during the training process as a learning curve into a file `learning_curve.txt` and visualize it.

End the training if a predefined number of training steps has been performed, and then test the performance of the network (no further weight changes) with respect to a second set of data, the test set `test_data.txt` (same file format as the training data). Choose resonable test data on your own.

The `training_data.txt` file starts with two lines of header, followed by P lines of data.

Each header line starts with a `#` character followed by some characters and strings that you can ignore (if you want to).

Each of the P data lines contains the data for one pattern p: N-input values ${}^p x_1, \ \ldots, \ {}^p x_N$, separated by one or more blanks, M-teacher values ${}^p \hat{y}_1, \ \ldots, \ {}^p \hat{y}_M$, separated by blanks.

Extra: (no extra points) If you are experienced with neural networks and MLPs, you can implement and train an MLP with Rectified Linear Units (ReLUs) using the *ramp function*.

In addition, make sure that your program is running correctly, is producing the required results, and that your source code contains valid, and useful comments.